

LAB: Create Your Own RAG AI Query

Course / project: SYSEN AIDATA (dsai) | **Folder:** 07_rag | **Submission files:** *clinical_rag_rules.yaml*, *06_patients_rag.py* (only)

Brief explanation (submission text)

Data source: Synthetic SQLite database *patients.db* (generated with *fakedata.py*) containing *patients* and *visits* tables with PHQ-9 scores, diagnoses, providers, and medications.

Search / retrieval: Parameterized SQL queries return exact rows or aggregates (counts, filters by PHQ-9 threshold and days since last visit). For the lapsed-follow-up query, cohort summary statistics are computed in Python and included in the JSON. Results are passed to the local LLM (Ollama).

System prompt: Rules in *clinical_rag_rules.yaml* (loaded into the system message) require a clinical, professional tone and prohibit inventing facts not present in the retrieval payload.

Source: *clinical_rag_rules.yaml*

System rules (YAML) — clinical tone and anti-hallucination constraints

```
# Clinical RAG assistant – system behavior (loaded by 06_patients_rag.py)
# Synthetic educational data only; not for real clinical decisions.

metadata:
  data_notice: >
  Context is retrieved from a synthetic SQLite database for coursework.
  Do not treat as real PHI or use for clinical decisions.

style:
  tone: clinical and professional
  language: clear, concise, neutral; avoid sensational wording
  format_preference: short headings and bullet points when listing facts

accuracy:
  - Use ONLY facts present in the retrieval JSON provided in the user message.
  - Do not invent patient names, IDs, dates, scores, diagnoses, medications, or counts
  .
  - If the retrieval is empty or a field is missing, say so explicitly instead of guessing.
  - Do not extrapolate beyond what the data supports (e.g., do not infer suicide risk without documented evidence in the retrieval).
  - Prefer stating "not available in the retrieved records" over filling gaps.

hallucination_prevention:
  - Treat the retrieval payload as the sole source of truth for this turn.
  - Do not cite external guidelines, trials, or dosing unless the user explicitly asks and retrieval does not apply.
  - If asked for information not in the retrieval, refuse briefly and restate what was provided.

clinical_safety:
  - This assistant summarizes retrieved records; it does not provide individualized medical advice.
  - Encourage appropriate follow-up with licensed clinicians for real-world care decisions.

privacy:
  - Do not request or assume real patient identifiers beyond what appears in the synthetic dataset.
```

Source: 06_patients_rag.py

Main RAG workflow — SQLite retrieval + JSON to LLM

```
# 06_patients_rag.py
# Custom RAG workflow: SQLite patients.db + Ollama (LAB_custom_rag_query.md)
# Tim Fraser (pattern); clinical retrieval + YAML-grounded system prompt.

# Retrieves structured rows from SQLite, passes them as JSON to the LLM for
# professional synopsis only—counts and filters come from SQL, not the model.

# 0. SETUP #####

## 0.1 Load Packages #####

import argparse
import json
import os
import runpy
import sqlite3
from typing import Optional

import pandas as pd

try:
    import yaml
except ImportError:
    yaml = None # optional; see load_rules()

from functions import agent_run

## 0.2 Working directory (script folder) #####

script_dir = os.path.dirname(os.path.abspath(__file__))
os.chdir(script_dir)

## 0.3 Start Ollama (same pattern as 04_sqlite.py) #####

ollama_script_path = os.path.join(script_dir, "01_ollama.py")
_ = runpy.run_path(ollama_script_path)

## 0.4 Configuration #####

MODEL = "gemma3"
PORT = 11434
OLLAMA_HOST = f"http://localhost:{PORT}"

DB_PATH = os.path.join(script_dir, "patients.db")
RULES_PATH = os.path.join(script_dir, "clinical_rag_rules.yaml")

# 1. LOAD YAML RULES → SYSTEM PROMPT STRING #####

def load_rules(path: str) -> str:
    """
    Build a single system prompt from clinical_rag_rules.yaml.
    Falls back to a short inline prompt if PyYAML is not installed.
    """
    if yaml is None or not os.path.isfile(path):
        return (
            "You are a clinical documentation assistant. Summarize ONLY the JSON "
            "facts in the user message. Do not invent names, dates, scores, or counts. "
            "If data are missing, say so. Professional, concise tone."
        )
    with open(path, encoding="utf-8") as f:
        cfg = yaml.safe_load(f)
        parts = []
        if cfg.get("metadata", {}).get("data_notice"):
            parts.append("Data notice: " + cfg["metadata"]["data_notice"].strip())
        if cfg.get("style"):
            s = cfg["style"]
            parts.append(
                f"Style: {s.get('tone', '')}. Language: {s.get('language', '')}. "
                f"Format: {s.get('format_preference', '')}."
            )
```

```

)
for key in ("accuracy", "hallucination_prevention", "clinical_safety", "privacy"):
    items = cfg.get(key)
    if isinstance(items, list):
        parts.append(key.upper().replace("_", " ") + ":")
        for line in items:
            parts.append(f"- {line}")
    return "\n".join(parts)

```

```

SYSTEM_PROMPT = load_rules(RULES_PATH)

```

```

# LLM instruction for Q4 (lapsed follow-up + cohort stats in JSON)
Q4_TASK = (

```

```

"Summarize overdue follow-up for this cohort using ONLY retrieval and "
"summary_statistics. Include: (1) how many patients and notable names/IDs if helpful; "
"(2) diagnosis_counts_at_last_visit – describe distribution with exact counts; "
"(3) mean_days_since_last_visit and median_days_since_last_visit from summary_statistics; "
"(4) last_visit_provider_counts – if all_lapsed_patients_share_same_last_provider is true, "
"clearly flag that single provider by name; otherwise highlight the most common last-visit "
"providers. Do not invent numbers or providers not in the JSON."
)

```

```

# 2. SEARCH / RETRIEVAL FUNCTIONS (SQLite) #####

```

```

def get_connection():
    """Open SQLite connection to patients.db."""
    if not os.path.isfile(DB_PATH):
        raise FileNotFoundError(
            f"Missing {DB_PATH}. Run fakerdata.py from the 07_rag folder first."
        )
    return sqlite3.connect(DB_PATH)

```

```

def retrieve_provider_patient_count(conn: sqlite3.Connection, provider: str) -> pd.DataFrame:
    """
    Distinct patients who have at least one visit with the given provider.
    """
    sql = """
    SELECT COUNT(DISTINCT patient_id) AS distinct_patient_count
    FROM visits
    WHERE provider = ?
    """
    return pd.read_sql_query(sql, conn, params=(provider,))

```

```

def retrieve_patient_care_records(conn: sqlite3.Connection, patient_id: int) -> pd.DataFrame:
    """Patient demographics + all visits for synopsis."""
    sql = """

```

```

SELECT
p.id AS patient_id,
p.name AS patient_name,
p.date_of_birth AS date_of_birth,
v.id AS visit_id,
v.visit_date,
v.phq9_score,
v.safety_concerns,
v.diagnosis,
v.provider,
v.medications
FROM patients p
LEFT JOIN visits v ON v.patient_id = p.id
WHERE p.id = ?
ORDER BY v.visit_date
"""
    return pd.read_sql_query(sql, conn, params=(patient_id,))

```

```

def retrieve_high_risk_phq9(conn: sqlite3.Connection, threshold: int = 15) -> pd.DataFrame:
    """Visits with PHQ-9 strictly greater than threshold (e.g., high symptom burden)."""
    sql = """
    SELECT
    p.id AS patient_id,
    p.name AS patient_name,
    v.visit_date,
    v.phq9_score,
    v.diagnosis,
    v.provider,
    v.safety_concerns
    FROM visits v
    JOIN patients p ON p.id = v.patient_id
    WHERE v.phq9_score > ?
    ORDER BY v.phq9_score DESC, v.visit_date DESC
    """
    return pd.read_sql_query(sql, conn, params=(threshold,))

def retrieve_not_seen_days(conn: sqlite3.Connection, min_days: int = 60) -> pd.DataFrame:
    """
    Patients whose most recent visit is more than min_days before today (ISO dates).
    Includes diagnosis and provider from that last visit for cohort summaries.
    """
    sql = """
    SELECT
    p.id AS patient_id,
    p.name AS patient_name,
    v.visit_date AS last_visit_date,
    v.diagnosis AS last_visit_diagnosis,
    v.provider AS last_visit_provider,
    CAST(julianday('now') - julianday(v.visit_date) AS INTEGER) AS days_since_last_v
    isit
    FROM patients p
    JOIN visits v ON v.patient_id = p.id
    WHERE v.id = (
    SELECT v2.id FROM visits v2
    WHERE v2.patient_id = p.id
    ORDER BY v2.visit_date DESC, v2.id DESC
    LIMIT 1
    )
    AND CAST(julianday('now') - julianday(v.visit_date) AS INTEGER) > ?
    ORDER BY last_visit_date ASC
    """
    return pd.read_sql_query(sql, conn, params=(min_days,))

def summarize_lapsed_cohort(df: pd.DataFrame) -> dict:
    """
    Aggregate stats for patients overdue for follow-up (computed in Python/SQL, not th
    e LLM).
    """
    if df.empty:
        return {
            "patient_count": 0,
            "mean_days_since_last_visit": None,
            "median_days_since_last_visit": None,
            "diagnosis_counts_at_last_visit": {},
            "last_visit_provider_counts": {},
            "all_lapsed_patients_share_same_last_provider": False,
            "single_last_provider_name_if_applicable": None,
            "note": "Empty cohort.",
        }

```

```

}

mean_d = float(df["days_since_last_visit"].mean())
med_d = float(df["days_since_last_visit"].median())
diag_counts = {
    str(k): int(v) for k, v in df["last_visit_diagnosis"].value_counts().items()
}
prov_counts = {
    str(k): int(v) for k, v in df["last_visit_provider"].value_counts().items()
}
}

```

```

n_unique = df["last_visit_provider"].nunique()
single_provider = n_unique == 1
single_name = str(df["last_visit_provider"].iloc[0]) if single_provider else None

return {
    "patient_count": int(len(df)),
    "mean_days_since_last_visit": round(mean_d, 1),
    "median_days_since_last_visit": round(med_d, 1),
    "diagnosis_counts_at_last_visit": diag_counts,
    "last_visit_provider_counts": prov_counts,
    "all_lapsed_patients_share_same_last_provider": bool(single_provider),
    "single_last_provider_name_if_applicable": single_name,
    "note": (
        "mean/median days_since_last_visit summarize how long each patient's last
        "
        "documented visit was ago (follow-up gap). This is not a count of missed a
        ppointments."
    ),
}

# 3. RAG: RETRIEVAL JSON + LLM SYNOPSIS #####

def rag_synopsis(
    role: str,
    task_label: str,
    df: pd.DataFrame,
    model: str = MODEL,
    summary_statistics: Optional[dict] = None,
) -> str:
    """

```

Pass retrieval as JSON; ask model for a synopsis grounded in that payload only.
Optional summary_statistics (e.g. cohort aggregates) are included when provided.

```

    """
    payload = {
        "task": task_label,
        "retrieval": df.to_dict(orient="records"),
        "row_count": len(df),
    }
    if summary_statistics is not None:
        payload["summary_statistics"] = summary_statistics
    task_json = json.dumps(payload, indent=2)
    return agent_run(role=role, task=task_json, model=model, output="text")

def demo_pick_provider(conn: sqlite3.Connection) -> str:
    """One provider from DB for the demo query."""
    row = conn.execute(
        "SELECT provider FROM visits ORDER BY provider LIMIT 1"
    ).fetchone()
    return row[0] if row else "Dr. Nobody"

def demo_pick_patient_id(conn: sqlite3.Connection) -> int:
    """First patient id for demo synopsis."""
    row = conn.execute("SELECT id FROM patients ORDER BY id LIMIT 1").fetchone()
    return int(row[0]) if row else 1

def run_all_demos(
    conn: sqlite3.Connection,
    model: str,
    provider: Optional[str] = None,
    patient_id: Optional[int] = None,
) -> None:
    """Run four lab queries: retrieval + LLM synopsis for each."""
    provider = provider or demo_pick_provider(conn)
    pid = patient_id if patient_id is not None else demo_pick_patient_id(conn)

# --- Q1: provider census ---
df1 = retrieve_provider_patient_count(conn, provider)
print("=" * 60)
print("Q1: How many distinct patients does a given provider have?")

```

```

print(f" (demo provider: {provider})")
print("SQL retrieval:", df1.to_dict(orient="records"))
out1 = rag_synopsis(
SYSTEM_PROMPT,
"Summarize how many distinct patients are associated with the specified provider "
"using ONLY the retrieval. One short paragraph plus a bullet with the count.",
df1,
model=model,
)
print(out1)
print()

# --- Q2: one patient synopsis ---
df2 = retrieve_patient_care_records(conn, pid)
print("=" * 60)
print("Q2: Synopsis of care for one patient")
print(f" (demo patient_id: {pid})")
out2 = rag_synopsis(
SYSTEM_PROMPT,
"Provide a brief, professional synopsis of this patient's care trajectory "
"using ONLY the retrieval (visits, PHQ-9, diagnoses, meds). "
"If there are no visits, say so.",
df2,
model=model,
)
print(out2)
print()

# --- Q3: PHQ-9 > 15 ---
df3 = retrieve_high_risk_phq9(conn, threshold=15)
print("=" * 60)
print("Q3: High-risk visits (PHQ-9 > 15) – synopsis of this cohort")
print(f" (rows retrieved: {len(df3)})")
out3 = rag_synopsis(
SYSTEM_PROMPT,
"Summarize this search: list themes (e.g., how many rows, date spread) using "
"ONLY the retrieval. Do not invent totals beyond row_count and the records.",
df3,
model=model,
)
print(out3)

```

```

print()

# --- Q4: not seen in > 60 days (with cohort summary stats in JSON) ---
df4 = retrieve_not_seen_days(conn, min_days=60)
lapsed_summary = summarize_lapsed_cohort(df4)
print("=" * 60)
print("Q4: Patients not seen in over 60 days (since last visit)")
print(f" (rows retrieved: {len(df4)})")
print(
" cohort aggregates (in JSON to LLM): mean_days="
f"{lapsed_summary.get('mean_days_since_last_visit')}, "
f"median_days={lapsed_summary.get('median_days_since_last_visit')}, "
f"same_last_provider_all={lapsed_summary.get('all_lapsed_patients_share_same_last_provider')}"
)
out4 = rag_synopsis(
SYSTEM_PROMPT,
Q4_TASK,
df4,
model=model,
summary_statistics=lapsed_summary,
)
print(out4)
print()

# 4. CLI: demo vs interactive parameters #####

def main():
    parser = argparse.ArgumentParser(
        description="Clinical RAG on patients.db: SQL retrieval + Ollama synopsis."
    )
    parser.add_argument(
        "--mode",

```

```

choices=("demo", "interactive"),
default="demo",
help=(
    "demo: run all four queries (optional --provider / --patient-id). "
    "interactive: prompts for provider and patient id for Q1-Q2, then runs Q3-
    Q4."
),

```

```

)
parser.add_argument("--model", default=MODEL, help="Ollama model name")
parser.add_argument(
    "--provider",
    default=None,
    help="Provider name for Q1 (exact match to visits.provider); demo picks one if
    omitted",
)
parser.add_argument(
    "--patient-id",
    type=int,
    default=None,
    help="Patient id for Q2; demo uses first patient if omitted",
)
args = parser.parse_args()

conn = get_connection()
try:
    if args.mode == "interactive":
        print("Interactive mode: enter values (blank uses demo defaults).")
        prov = input("Provider name for Q1 (e.g. Dr. Smith): ").strip() or demo_pi
        ck_provider(
            conn
        )
        df1 = retrieve_provider_patient_count(conn, prov)
        print("\n--- Q1 ---\n", rag_synopsis(SYSTEM_PROMPT, "Q1", df1, args.model)
        )
        pid_s = input("Patient id for Q2: ").strip()
        pid = int(pid_s) if pid_s else demo_pick_patient_id(conn)
        df2 = retrieve_patient_care_records(conn, pid)
        print("\n--- Q2 ---\n", rag_synopsis(SYSTEM_PROMPT, "Q2", df2, args.model)
        )
        df3 = retrieve_high_risk_phq9(conn, 15)
        print("\n--- Q3 ---\n", rag_synopsis(SYSTEM_PROMPT, "Q3", df3, args.model)
        )
        df4 = retrieve_not_seen_days(conn, 60)
        s4 = summarize_lapsed_cohort(df4)
        print(
            "\n--- Q4 ---\n",
            rag_synopsis(
                SYSTEM_PROMPT,
                Q4_TASK,

```

```

df4,
args.model,
summary_statistics=s4,
),
)
else:
    run_all_demos(
        conn,
        args.model,
        provider=args.provider,
        patient_id=args.patient_id,
    )
finally:
    conn.close()

if __name__ == "__main__":
    main()

```


Console output — answers to the four queries

Below is the terminal output from running `python 06_patients_rag.py` (demo mode). SQL retrieval lines print where applicable; Q4 also prints cohort aggregate hints. Narrative answers are model-generated summaries grounded in the JSON payload (and `summary_statistics` for Q4).

```
=====
Q1: How many distinct patients does a given provider have?
(demo provider: Dr. Aguilar)
SQL retrieval: [{'distinct_patient_count': 2}]
This provider is associated with two distinct patients based on the available records.

* Distinct Patient Count: 2

=====
Q2: Synopsis of care for one patient
(demo patient_id: 1)
**Patient Synopsis: Justin Miller**

* **Initial Assessment:** Mr. Miller's care began with a diagnosis of Panic Disorder (visit ID 1) with an associated PHQ-9 score of 4 and medication of Adderall prescribed by Dr. Smith.
* **Anxiety and Depression:** Subsequent visits (visit IDs 2, 3, 4, and 5) revealed a fluctuating presentation. Initially, Generalized Anxiety Disorder was diagnosed (visit ID 2) with a PHQ-9 score of 8 and Mirtazapine prescribed by Dr. Pierce. A diagnosis of Persistent Depressive Disorder was later established (visits ID 3 and 4) with PHQ-9 scores of 5 and 6, respectively, and treatment with Escitalopram, Vyvanse, Venlafaxine, and Mirtazapine prescribed by Dr. Hutchinson and Dr. Campbell.
* **Current Diagnosis:** At the last recorded visit (visit ID 5), a diagnosis of Major Depressive Disorder was noted with a PHQ-9 score of 6 and medications of Lamotrigine, Vyvanse, and Mirtazapine prescribed by Dr. Collier.

No visits were recorded prior to February 27, 2026.

=====
Q3: High-risk visits (PHQ-9 > 15) – synopsis of this cohort
(rows retrieved: 135)
Here's a summary of the provided search data:

* **Total Records:** 135
* **Patient Data:** The records include information for 83 unique patients.

**Patient Diagnoses and PHQ9 Scores:**

* **ADHD:** Kevin Holmes (51) appears in 3 records with a PHQ9 score of 27, 23, and 22. Scott Meyer (17) appears in 2 records with a PHQ9 score of 20 and 16.
* **Generalized Anxiety Disorder:** Max Wilson (2) appears in 1 record with a PHQ9 score of 27. Daniel Smith (75) appears in 2 records with a PHQ9 score of 25 and 21.

* **Social Anxiety Disorder:** Kevin Holmes (51) appears in 3 records with a PHQ9 score of 27, 22, and 16. Sharon Ramsey (46) appears in 4 records with a PHQ9 score of 22, 22, 17 and 16.
* **Bipolar II Disorder:** Robert Hutchinson (37) appears in 3 records with a PHQ9 score of 26, 22, and 16. Amber Ferguson (47) appears in 2 records with a PHQ9 score of 22 and 16.
* **Persistent Depressive Disorder:** Danielle Ward (24) appears in 1 record with a PHQ9 score of 26.
* **Obsessive-Compulsive Disorder:** Kevin Diaz (4) appears in 3 records with a PHQ9 score of 24, 22, and 16. Christian Cole (32) appears in 1 record with a PHQ9 score of 20.
* **Panic Disorder:** Max Wilson (2) appears in 1 record with a PHQ9 score of 27. John Molina (71) appears in 2 records with a PHQ9 score of 21 and 16.
* **PTSD:** Robert Hutchinson (37) appears in 1 record with a PHQ9 score of 25. Natasha Anderson (65) appears in 2 records with a PHQ9 score of 22 and 16.
* **Major Depressive Disorder:** Tammy Williamson (54) appears in 2 records with a PHQ9 score of 25 and 17. Morgan Livingston (14) appears in 1 record with a PHQ9 score of 20.
* **Specific Scores:** Several individuals have a PHQ9 score of 23.

**Providers:**

* The records involve the following providers: Dr. Fuller, Dr. Brooks, Dr. Mitchell, Dr. Sims, Dr. Aguilar, Dr. Little, Dr. Rivas, Dr. Mcgrath, Dr. Nguyen, Dr. Black, Dr. McKee, Dr. Arnold, Dr. Patel, Dr. Yu, Dr. Perez, Dr. Hardin, Dr. Garcia, Dr. Murphy, Dr. Olsen, Dr. Gomez, Dr. Stewart, Dr. Green, Dr. Watson, Dr. Lee, Dr. Rogers, Dr. Hal
```

l, Dr. Hudson, Dr. Miller, Dr. Ramirez, Dr. Carter, Dr. Wright, Dr. Freeman, Dr. Stone
, Dr. Berry, Dr. Valenzuela, Dr. Jacobs, Dr. Rodgers, Dr. Chen, etc.

=====

Q4: Patients not seen in over 60 days (since last visit)
(rows retrieved: 69)
cohort aggregates (in JSON to LLM): mean_days=240.3, median_days=209.0, same_last_
provider_all=False
Overdue Follow-Up Summary

This summary identifies patients with overdue follow-up appointments based on the provided retrieval data.

* **Patient Count:** 69 patients have an overdue follow-up appointment.
* **Notable Patients:**
* Mark Mendoza (Patient ID: 91) was last seen on 2024-07-06.

* Wendy Jackson (Patient ID: 27) was last seen on 2024-07-23.
* Valerie Murphy (Patient ID: 48) was last seen on 2024-08-26.
* Charles Cross (Patient ID: 79) was last seen on 2024-09-24.
* Morgan Livingston (Patient ID: 14) was last seen on 2024-10-16.
* Jon Holland (Patient ID: 80) was last seen on 2024-12-07.
* Sarah Duran (Patient ID: 21) was last seen on 2025-01-20.
* Kathryn Johnson (Patient ID: 82) was last seen on 2025-02-17.
* Gilbert Roberts (Patient ID: 25) was last seen on 2025-02-28.
* Emily Price (Patient ID: 39) was last seen on 2025-03-11.
* William Riley (Patient ID: 100) was last seen on 2025-03-15.
* Krista Wilcox (Patient ID: 9) was last seen on 2025-03-16.
* Justin Hall (Patient ID: 89) was last seen on 2025-03-19.
* Laurie Chavez (Patient ID: 93) was last seen on 2025-03-21.
* Jerry Morris (Patient ID: 13) was last seen on 2025-04-03.
* Paul Murphy (Patient ID: 72) was last seen on 2025-04-08.
* Christian Cole (Patient ID: 32) was last seen on 2025-04-19.
* Gregory Robertson (Patient ID: 16) was last seen on 2025-05-22.
* Stephanie Ross (Patient ID: 62) was last seen on 2025-06-03.
* Katherine Edwards (Patient ID: 64) was last seen on 2025-06-23.
* Joseph Robertson (Patient ID: 78) was last seen on 2025-06-26.
* Maria Ramos (Patient ID: 74) was last seen on 2025-06-23.
* Douglas McKenzie (Patient ID: 33) was last seen on 2025-06-26.
* Kyle Berger (Patient ID: 78) was last seen on 2025-06-26.
* Stephanie Dominguez (Patient ID: 94) was last seen on 2025-06-27.
* Scott Meyer (Patient ID: 17) was last seen on 2025-07-01.
* Marie Jensen (Patient ID: 76) was last seen on 2025-07-14.
* Diana Hall (Patient ID: 20) was last seen on 2025-07-16.
* Max Wilson (Patient ID: 2) was last seen on 2025-07-27.
* Gina Schwartz (Patient ID: 70) was last seen on 2025-07-31.
* Joshua Johnson (Patient ID: 55) was last seen on 2025-08-10.
* Rachel Nelson (Patient ID: 6) was last seen on 2025-08-10.
* Kevin Kim (Patient ID: 51) was last seen on 2025-08-10.
* Nicole Clay (Patient ID: 43) was last seen on 2025-08-25.
* Bronwyn Howard (Patient ID: 53) was last seen on 2025-08-28.
* Alicia Ross (Patient ID: 68) was last seen on 2025-09-04.
* Dominic Koch (Patient ID: 87) was last seen on 2025-09-26.
* Thomas Cardenas (Patient ID: 83) was last seen on 2025-09-27.
* Lauren Barrett (Patient ID: 63) was last seen on 2025-09-28.
* David Long (Patient ID: 15) was last seen on 2025-09-04.
* Ashley Bass (Patient ID: 23) was last seen on 2025-09-05.
* Jordan Carter (Patient ID: 71) was last seen on 2025-09-06.
* Harry Taylor (Patient ID: 50) was last seen on 2025-09-08.

* Dr. Harry Taylor (Patient ID: 50) was last seen on 2025-09-08.
* Sean Bell (Patient ID: 51) was last seen on 2025-09-08.
* Eric White (Patient ID: 56) was last seen on 2025-09-08.
* Richard Mcknight (Patient ID: 3) was last seen on 2025-09-26.

* **Diagnosis Distribution:**
* Social Anxiety Disorder: 9 patients
* ADHD: 9 patients
* Bipolar II Disorder: 9 patients
* Major Depressive Disorder: 8 patients
* Panic Disorder: 8 patients
* Persistent Depressive Disorder: 6 patients
* Obsessive-Compulsive Disorder: 5 patients
* PTSD: 4 patients
* Generalized Anxiety Disorder: 11 patients

* **Days Since Last Visit:**

* Mean days since last visit: 240.3 days
* Median days since last visit: 209 days

* **Last Visit Providers:**

* Dr. Davis: 3 patients
* Dr. Bell: 2 patients
* Dr. Murphy: 1 patient
* Dr. Simmons: 1 patient
* Dr. Brown: 1 patient

Note: The mean and median days since last visit summarize the average time since the patient's last documented appointment. This data does not represent missed appointments.